

Evaluating the Efficacy of Next.js: A Comparative Analysis with React.js on Performance, SEO, and Global Network Equity

Swostik Pati

New York University Abu Dhabi
Abu Dhabi, United Arab Emirates
swostik.pati@nyu.edu

Yasir Zaki

New York University Abu Dhabi
Abu Dhabi, United Arab Emirates
yasir.zaki@nyu.edu

Abstract

This paper investigates the efficacy of Next.js as a framework addressing the challenges posed by React.js, particularly in performance, SEO, and equitable web accessibility. By constructing identical websites and web applications in both frameworks, we aim to evaluate the frameworks' behavior under diverse network conditions and capabilities. Beyond quantitative metrics like First Contentful Paint (FCP) and Time to Interactive (TTI), we incorporate qualitative user feedback to assess real-world usability. Our motivation stems from bridging the digital divide exacerbated by client-side rendering (CSR) frameworks and validating investments in modern technologies for businesses and institutions. Employing a novel LLM-assisted migration workflow, this paper also demonstrates the ease with which developers can transition from React.js to Next.js. Our results highlight Next.js's promise of better overall performance, without any degradation in user interaction experience, showcasing its potential to mitigate disparities in web accessibility and foster global network equity, thereby emerging as a compelling framework for shaping an inclusive web.

CCS Concepts

• Information systems → World Wide Web; Web interfaces.

Keywords

Next.js, React.js, Client/Server Side Rendering, SEO optimization

ACM Reference Format:

Swostik Pati and Yasir Zaki. 2025. Evaluating the Efficacy of Next.js: A Comparative Analysis with React.js on Performance, SEO, and Global Network Equity. In *Companion Proceedings of the ACM Web Conference 2025 (WWW Companion '25)*, April 28-May 2, 2025, Sydney, NSW, Australia. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3701716.3715565>

1 Introduction

In the dynamic realm of web development, the transition from traditional server-side rendering (SSR) to client-side rendering (CSR) has marked a significant evolution in how web applications are designed and interacted with. SSR operates by having the webpage's content be first generated on the server and then sent over the

network to the user's browser in its final HTML form. This method, while stable and reliable, often results in slower response times to user interaction, as each request necessitates a new page to be served from the server. To counter this, CSR was introduced as a more modern approach that shifted the rendering operations to the browser. Web applications download a minimal HTML page from the server, which then uses loads of JavaScript files to generate the rest of the page's content dynamically. This shift not only reduces the server load but also significantly enhances the interactivity and fluidity of user interfaces. Web apps became more responsive, providing a smoother user experience as the browser could re-render only specific parts of the page in response to user actions without needing to fetch a new page from the server each time.

However, as the landscape evolved, it became evident that CSR, despite its improvements in reactivity and user interface fluidity, has introduced significant challenges. These include suboptimal search engine optimization (SEO), poorer performance [12], and an increased demand on the client resources. As such, it led on to a disparate web experience across varied network conditions, highlighting a digital divide that is more pronounced in low-bandwidth environments. One major example of this is the case of React.js [19], which exemplifies both the strengths and weaknesses of CSR. React.js is a client-side JavaScript framework that was widely adopted around the world ever since its launch and is currently the second most used web framework with almost 40% market share [17]. However, even with such wide adoption rates, frequent updates, and several workarounds and optimizations, React.js still suffers from all the aforementioned issues and challenges of CSR.

All of the above has led to an increased interest in what are known as hybrid frameworks, such as Next.js [20] (a JS framework built over React), which mitigates these issues by combining the best features of SSR and CSR. Next.js promises to enhance SEO and performance through techniques like server-side rendering, pre-fetching, incremental static regeneration, and static site generation. In addition, through its default caching mechanisms and use of server components (which reduce the client-side overhead), it optimizes resource utilization, ensuring that users worldwide, regardless of their network, receive a faster and seamless experience.

Due to the benefits of Next.js, the web development world has been fast to adopt this framework, with Next.js having become the 4th most used web framework globally (i.e., 18% market share [17]) in less than ten years since its inception. Given the widespread migration towards Next.js, this study seeks to preemptively address potential pitfalls by evaluating Next.js and benchmarking its performance against its parent library, React.js, across various performance metrics (i.e., loading times, lighthouse scores), network

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
WWW Companion '25, Sydney, NSW, Australia

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1331-6/25/04
<https://doi.org/10.1145/3701716.3715565>

conditions, and device processing capabilities. By doing so, we set out to understand whether Next.js can truly live up to its promises.

While Next.js has been widely discussed for its advantages over traditional client-side rendering (CSR) frameworks like React.js, prior works primarily focus on theoretical or isolated performance benchmarks. Our study is novel in that it systematically evaluates Next.js across multiple real-world network conditions and device constraints, incorporating both quantitative metrics and qualitative feedback. We also introduce a LLM-assisted migration workflow to assess the feasibility of transitioning from React.js to Next.js at scale. Finally, we examine Next.js's role in bridging network disparities, a perspective that is largely overlooked in prior research.

In this paper we answer the following questions: **RQ1** Does Next.js have an overall faster performance compared to React.js across different network conditions and devices? **RQ2** Does Next.js have better Search Engine Optimization (SEO) metrics than React.js? **RQ3** Beyond the quantitative evaluations, does Next.js match—or not exceed—React.js in terms of user interaction experience?

2 Motivation

The motivation behind this work is twofold: ensuring that the world as a whole gets to experience the web in a similar way, bridging the digital divide that is currently present due to disparities across network bandwidths and device processing capabilities [4]; and ensuring that when companies migrate into newer web technologies such as Next.js, they are actually worth the investment.

2.1 Towards Global Network equity

The enhanced interactivity offered by CSR frameworks like React.js in regions with robust network connectivity can result in significantly longer loading times in areas with limited bandwidth [12, 16]. This disparity in performance can disproportionately affect users in regions with poor network infrastructure. This discrepancy arises because CSR frameworks transfer the entirety of the webpage data to the client side for processing [3, 12], necessitating users in low-bandwidth environments to download substantial amounts of data and render the application on devices with limited processing.

Koradia et al. [13] reported cellular data connectivity latencies in India reaching up to 1200 ms. When compounded by the data processing demands of CSR on the client side, such latencies can significantly amplify delays and diminish user experience.

Additionally, in bandwidth-constrained environments, TCP flows encounter high packet loss rates, severe unfairness, and repetitive timeouts, which could render CSR-based websites practically unusable [22]. Furthermore, by requiring low-end devices to process and render entire applications locally after downloading substantial data, CSR frameworks place a disproportionate burden on users in these regions. This challenge exacerbates the digital divide, as individuals with limited internet access and processing capabilities experience slower and less responsive websites, hindering their ability to participate in the digital economy [3, 11].

One of the main objectives behind this work is to ensure that the adoption of novel technologies like Next.js, particularly at scale, is conducted responsibly, promoting equitable access to digital resources. With features like SSR, static site generation (SSG), and caching by default, Next.js seems like a promising solution to reduce

client-side processing and improve performance for users in low-resource settings. However, it is essential to validate the efficacy of these assumptions through rigorous evaluation to ensure that its adoption addresses the challenges faced in developing regions.

2.2 Businesses and other Institutions

In 2006, Amazon showed that each 100ms increase in page load time correlated with a 1% reduction in sales, representing an annual loss of \$107 million—an impact equivalent to over \$3.8 billion today [6]. This study shows how critical web performance is to business success. However, the reliance on CSR introduces significant drawbacks, as the need for browsers to download and execute large JavaScript files can lead to slower initial page loads [14], frustrating users and increasing bounce rates, particularly among low-end devices. With studies showing that a 100ms delay can reduce conversion rates by 7% and 53% of mobile users abandon sites that take over three seconds to load [1], it becomes imperative that adopting newer technologies like Next.js to ensure these migrations deliver measurable benefits, especially in terms of faster performance and better search engine indexing [3]. Next.js holds the potential to drive better user experiences; however, validating these claims is essential to justify the significant investment in such transitions.

3 Evaluation Methodology

3.1 Quantitative Metrics Analysis

Here, we compare important performance metrics [10] relating to the different timing stages of loading a page, such as First Contentful Paint (FCP), Time to Interactive (TTI), Page Load Time (PLT), and Lighthouse SEO scores of sites built using both React.js and Next.js. One of the main challenges that we faced was finding a dataset that contains websites built using both React.js and Next.js. While some organizations may have migrated from one framework to the other, it is uncommon for both versions to remain operational and receive consistent maintenance. We had access to a large number of Next.js-only or React.js-only websites, but it was obvious that comparing unrelated websites would not provide meaningful insights. So to ensure a fair comparison, we created two sets of the same websites using both React.js and Next.js, which was challenging because Next.js uses a file-system-based router [15].

3.1.1 Defining the Testing Framework. First, we began by outlining the specific features offered by Next.js by default to determine which types of websites would be the most meaningful to create. The features that were centered around performance optimization were partial re-rendering, pre-fetching, soft navigation, code splitting, caching, parallel routes, intercepting routes, server actions, and incremental static regeneration [18]. On top of these, the element-level optimizations for images, videos, fonts, etc. could be additionally tested in any of the websites created.

As explained earlier, since the only way to obtain a dataset of equivalent websites was to create it ourselves, we made sure to rigorously choose a diverse set of applications, where some would target specific features, while others would test all-around performance, ensuring a comprehensive evaluation of real-world use cases. Seven pairs of websites and three pairs of web apps (with identical back-ends) were carefully chosen for this purpose. These

included a travel destination explorer, a to-do app, a Wikipedia article viewer, a real-time stock tracker, a GitHub project portfolio, a travel booking website, etc [18]. Many of these had additional integrations like embedded maps (leaflet), charting libraries like Recharts, styling libraries like Tailwind and Bootstrap, APIs capable of sourcing real-time stock analytics, GitHub projects, Wikipedia articles, etc. Two of the web apps were full-scale clones of popular ones—Amazon and YouTube. This allowed to perform a holistic assessment by testing the two frameworks under different conditions.

3.1.2 LLM-Assisted Migration Workflow. To make the process of creating the websites and web apps simpler, we decided on implementing an LLM-assisted migration workflow that involved first creating the websites on React.js and then using Claude-engineer to set up and migrate to a Next.js project. Claude-engineer [9], an advanced interactive command-line interface, utilizes the capabilities of the Claude 3.5 Sonnet LLM in conjunction with local system functions to access files, initiate projects, and execute code. It was well-trained on the Next.js documentation along with detailed migration instructions recommended by Next.js [21].

Although the initial draft of a Next.js project was generated using the LLM-assisted workflow, subsequent manual adjustments were necessary to fully leverage its optimization features. This refinement process required a significant time investment, but with fine-tuned LLM models specifically trained for this purpose, scalability will not be an issue. This is because Next.js is built over React.js and therefore several parts of the core codebase remain the same. We anticipate that such a flow of using the semi-automated LLM-assisted migration workflow can become the recommended workflow for similar migrations.

3.1.3 Performance Testing and Analysis. For the purpose of performance testing, we decided on using FCP and TTI in three different network conditions: 4G Fast (10 Mbps, 40ms RTT), 4G Slow (9 Mbps, 170ms RTT), and 3G (1.6 Mbps/768 Kbps 300ms RTT), under both normal and 6x lower CPU-throttled conditions to measure the initial page load metrics using the “Performance Window” in the Chrome Developer Tools [5]. Metrics such as FCP and TTI are vital as they directly reflect the user’s real-world experience of loading and interactivity [10]. These are industrial benchmarks to evaluate web pages. We do acknowledge that TTI has its limitations of being very sensitive to “outlier network requests and long tasks” [8], so we made sure to use it as an additional metric while at the same time keeping identical simulation conditions. We also measured the lighthouse SEOs [7]. Lighthouse SEO assesses a page based on its meta description, as SEO depends a lot on how fast search engine crawlers are able to pick up the text of the webpage and index it. This becomes an issue with purely CSR frameworks like React.js as search engines struggle to index dynamic content rendered on the client side, which hybrid frameworks like Next.js are able to solve.

To extensively evaluate the websites, we simulated identical sets of actions - for e.g., clicking something, searching something, navigating, etc. - on both versions and recorded the ‘Finish’ time in the network tab to determine the final loading time [2]. The ‘Finish’ time was unaffected by manual delays during the simulations, as it exclusively measured the total time taken for loading all the resources requested. However, due to limitations of the Network tab, we could not replicate these tests under throttled CPU conditions.

Furthermore, the network tab was paused at the end to prevent any more network requests before reading the ‘Finish’ time. This testing methodology was replicated across all three network conditions. However, inherent limitations of the network tab precluded the simulation of the throttled CPU conditions. To prevent any selector’s bias in sourcing these values, we made sure to take the median of three runs for every single metric across all the websites.

3.2 User-Centric Qualitative Assessment

In addition to the quantitative evaluation, we also incorporated a qualitative evaluation to gather user feedback on the usability and performance of the websites. This was achieved through participatory research studies, where a diverse group of participants from NYU Abu Dhabi were recruited. 67 participants with devices of varying specifications of GPU, RAM, and other hardware configurations were included to analyze the website performance under real-world contexts. They provided feedback based on their experiences with loading times, interaction response times, navigation smoothness, and the overall browsing experience.

The diversity of the NYU Abu Dhabi community, with students from over 100 countries studying across several disciplines, was chosen to bring varied perspectives and responses to the study. Two sets of surveys, where the ordering of the websites was changed to prevent bias, were sent out to the participants. The participants evaluated two full-scale clones of Amazon and YouTube created in both Next.js and React. While only two sets of web applications were tested, this decision was deliberate and necessary to ensure that the study could focus on the overall quality of user interaction in a robust and controlled manner. The selection of these apps was predicated on their capacity to rigorously evaluate the capabilities of Next.js, encompassing a diverse set of features representative of essential web development use cases. By selecting clones of globally significant apps with varying use cases and design approaches, we ensured that the insights drawn are not skewed by edge cases, allowing us to source statistically significant data and derive meaningful conclusions, ensuring the study validity and impact.

We made sure that the links and questions were presented as “Version 1” and “Version 2” to prevent bias based on prior understanding of the technologies. We also ensured that participants understood this study was not focused on the interface design or visual aesthetics but rather on evaluating the overall user experience, emphasizing speed and smoothness during website navigation.

4 Results

4.1 Quantitative Results

After obtaining the data points for each metric across all conditions, we calculated the delta time between the React and Next.js versions of each website for every metric. This delta time was computed as the difference between the corresponding values of React and Next.js (React - Next.js), providing a direct comparison of performance. Figures 1(a) and 1(b) show the cumulative distribution functions (CDFs) for each metric under all network conditions (fast 4G, slow 4G, and 3G) for both the normal CPU and throttled conditions, respectively. The results show that the delta values are positive for almost all metrics and network conditions, apart from a few outliers in the case of the TTI metric for fast and slow 4G

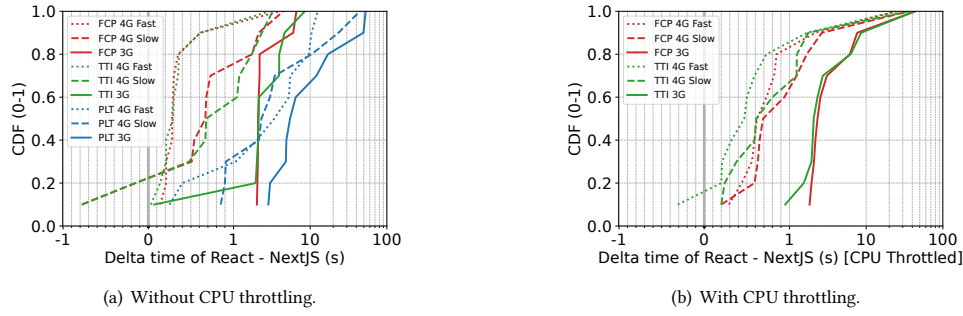


Figure 1: Quantitative evaluation results of Next.js vs. React.js across different network conditions.

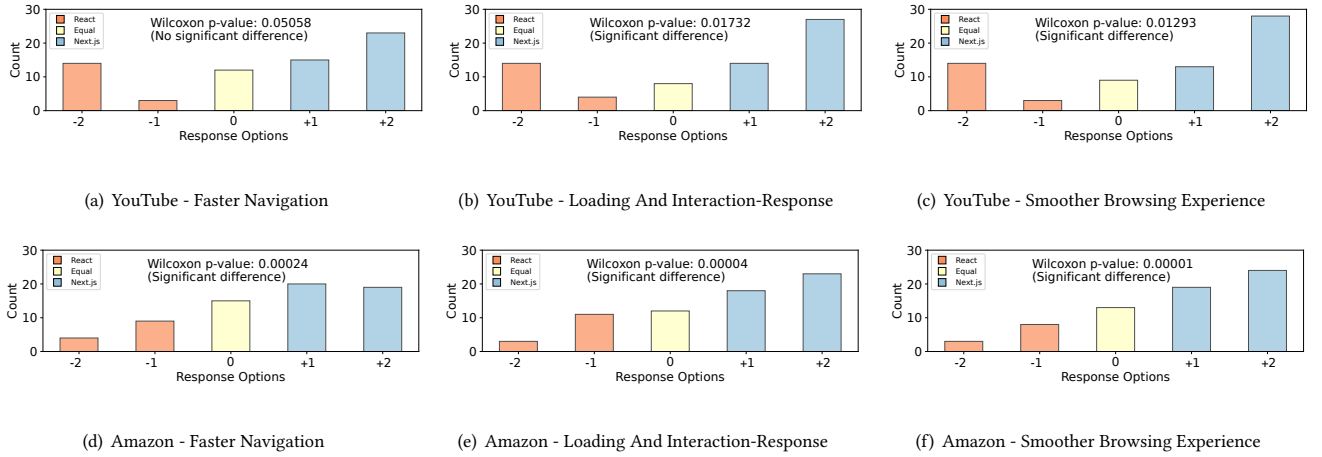


Figure 2: YouTube and Amazon clone websites quantitative evaluation.

(dotted and dashed green curves). Additionally, the results also highlight that with slower network connectivity, all timing metrics are improved in webpages built with Next.js compared to React, as well as with throttled CPUs. This highlights the significance of using Next.js to improve the overall performance for users in developing regions suffering from both poorer network connectivity and over-reliance on low-end mobile devices. Finally, for the SEO evaluations, Next.js received an average score of 100% compared to React, which received a score of 88.8%. Even though the websites had a very nominal amount of content, it still considerably impacted the SEO scores, with Next.js always getting a 100 and React.js fluctuating over a wide range of values. These results therefore help us conclusively answer both **RQ1**, that Next.js indeed has overall faster performance compared to React.js across different network conditions and device processing capabilities; and **RQ2**, that Next.js has better SEO metrics than React.js.

4.2 Qualitative Survey Insights

The survey received a total of 67 responses, representing approximately 30 distinct device configurations. For every question, the participants had an option to choose on a scale of 5 values: "Next.js/React.js was clearly better", "Next.js/ React.js was slightly better", and "Both are equivalent." We encoded these to integer values from -2 to +2, taking 0 (equivalent) as our null hypothesis. Then, using the

Wilcoxon p-value method, we ascertained which metrics showed statistically significant results. Figure 2 shows the response results, highlighting that participants preferred the Next.js version of the YouTube clone for the loading and interaction-response times and for the overall smoother browsing experience. The survey also showed that participants preferred the Next.js version of the Amazon clone for all three metrics: faster navigation, interaction times, and a smoother browsing experience. This answered **RQ3**, concluding that Next.js does offer a better user experience than React.js.

5 Conclusion

Our results, provide strong evidence that Next.js significantly improves the performance metrics, especially for poor network conditions, while maintaining an improved user interaction experience. Beyond these findings, the research highlighted the importance of equitable access to web resources, emphasizing Next.js's role in mitigating the digital divide and fostering global network equity. We lay the foundation for broader investigations into the scalability and adaptability of Next.js (and other hybrid frameworks) across a wider spectrum of use cases, including enterprise-level applications and globally distributed user bases. By expanding the scope to include larger datasets and more complex user scenarios, future research can deepen our understanding of Next.js's potential in shaping a more inclusive and high-performing digital ecosystem.

References

- [1] Akamai Technologies, Inc. 2017. *Akamai Releases Spring 2017 State of Online Retail Performance Report*. <https://www.akamai.com/newsroom/press-release/akamai-releases-spring-2017-state-of-online-retail-performance-report>
- [2] Kayce Basques and Sofia Emelianova. 2024. Network Features Reference - Chrome DevTools. <https://developer.chrome.com/docs/devtools/network/reference>
- [3] Half Nine Blog. 2024. *Server-Side vs Client-Side Rendering*. <https://www.halfnine.com/blog/post/server-side-vs-client-side>
- [4] Moumena Chaqfeh, Rohail Asim, Bedoor AlShebli, Muhammad Fareed Zaffar, Talal Rahwan, and Yasir Zaki. 2023. Towards a world wide web without digital inequality. *Proceedings of the National Academy of Sciences* 120, 3 (2023).
- [5] Chrome Developers. 2024. Performance Panel Reference - Chrome Developers. <https://developer.chrome.com/docs/devtools/performance/reference>
- [6] Conductor. 2024. *Amazon Page Speed Study*. <https://www.conductor.com/academy/page-speed-resources/faq/amazon-page-speed-study/>
- [7] Google Developers. 2024. Lighthouse Overview. <https://developer.chrome.com/docs/lighthouse/overview>
- [8] Google Chrome Developers. 2024. Time to Interactive (TTI) - Performance Metrics. <https://developer.chrome.com/docs/lighthouse/performance/interactive>
- [9] Doriandarko. 2024. Claude-Engineer: An advanced interactive CLI. <https://github.com/Doriandarko/claude-engineer>
- [10] Google Developers. 2020. *Core Web Vitals*. <https://web.dev/vitals/>
- [11] Ilya Grigorik. 2013. *High Performance Browser Networking*. O'Reilly Media, Inc., Sebastopol, CA. <https://hpbn.co/>
- [12] Taufan Iskandar, Muharman Lubis, Tien Kusumasari, and Arif Lubis. 2020. Comparison between client-side and server-side rendering in the web development. *IOP Conference Series Materials Science and Engineering* 801 (06 2020), 012136.
- [13] Zahir Koradia, Goutham Mannava, Aravindh Raman, Gaurav Aggarwal, Vinay Ribeiro, Aaditeshwar Seth, Sebastian Ardon, Anirban Mahanti, and Sipat Triukose. 2013. First impressions on the state of cellular data connectivity in India. In *Proceedings of the 4th Annual Symposium on Computing for Development* (Cape Town, South Africa) (*ACM DEV-4 '13*). New York, NY, USA, Article 3, 10 pages.
- [14] Jesutofunmi Kupoluyi, Moumena Chaqfeh, Matteo Varvello, Russell Coke, Waleed Hashmi, Lakshmi Subramanian, and Yasir Zaki. 2022. Muzeel: Assessing the impact of JavaScript dead code elimination on mobile web performance. In *Proceedings of the 22nd ACM Internet Measurement Conference*. 335–348.
- [15] Next.js Team. 2024. Layouts and Pages - Next.js Documentation. <https://nextjs.org/docs/app/getting-started/layouts-and-pages>
- [16] Carl Nordström and August Dixelius. 2023. Comparisons of Server-side Rendering and Client-side Rendering for Web Pages. (2023). <https://uu.diva-portal.org/smash/get/diva2:1775555/FULLTEXT02.pdf>
- [17] Stack Overflow and Statista. 2024. Most Used Web Frameworks Among Developers Worldwide, as of 2024. Statista. <https://www.statista.com/statistics/1124699/worldwide-developer-survey-most-used-frameworks-web/>
- [18] Swostik Pati. 2024. Next.js vs React.js. <https://swostikpati.notion.site/nextjs-reactjs>
- [19] React Team. 2024. *React - A JavaScript library*. <https://react.dev/>
- [20] Vercel. 2024. *Next.js - The React Framework for Production*. <https://nextjs.org/>
- [21] Vercel. 2024. Upgrading from Vite to Next.js App Router. <https://nextjs.org/docs/app/building-your-application/upgrading/from-vite>
- [22] Yasir Zaki, Jay Chen, Thomas Pötsch, Talal Ahmad, and Lakshminarayanan Subramanian. 2014. Dissecting Web Latency in Ghana. In *Proceedings of the 2014 Conference on Internet Measurement Conference* (Vancouver, BC, Canada) (*IMC '14*). Association for Computing Machinery, New York, NY, USA, 241–248.